SOFTWARE REQUIREMENTS AND DESIGN DOCUMENT

CAMPUSGPT — PDF RAG CHATBOT

An AI-Powered PDF Question Answering System

Minor Project

Submitted by:

JAKKA KRUPA RATNAM

ABSTRACT

CampusGPT — PDF RAG Chatbot is an AI-powered web application designed to help users obtain information from PDF documents through natural-language questions. Searching through lengthy academic documents manually can be time-consuming, particularly when users need specific information quickly.

The proposed system uses Retrieval-Augmented Generation (RAG) to combine document retrieval with AI-based response generation. When a PDF is uploaded, the system extracts its textual content and divides it into smaller chunks. These chunks are converted into numerical vector representations using a Sentence Transformer embedding model and stored in a FAISS vector index.

When the user submits a question, the system converts the question into an embedding and performs similarity search against the document vectors. The most relevant document chunks are retrieved and provided as context to the Gemini language model. Gemini then generates a natural-language response based on the retrieved context.

The application is developed using Python and Flask, with HTML, CSS, and JavaScript used for the web interface. PyPDF is used for PDF text extraction, Sentence Transformers are used for embeddings, FAISS is used for vector similarity search, and Gemini is used for AI-based response generation.

The project demonstrates how Retrieval-Augmented Generation can be applied to educational documents to provide a simple and interactive PDF question-answering experience.

1. INTRODUCTION

1.1 Purpose

CampusGPT is an AI-powered PDF question-answering system designed to help users obtain relevant information from uploaded PDF documents.

The system uses a Retrieval-Augmented Generation (RAG) approach. Instead of asking the language model to answer questions using only its general knowledge, CampusGPT first searches the uploaded document for relevant information and then provides the retrieved information as context to the Gemini language model.

The main purpose of the system is to provide a simple web-based interface where a user can upload a PDF document and ask questions about its contents.

1.2 Project Overview

CampusGPT is developed as a web application using Python and Flask.

The system processes an uploaded PDF through several stages:

1. PDF text extraction
2. Text chunking
3. Embedding generation
4. Vector indexing using FAISS
5. Similarity-based retrieval
6. Context construction
7. Answer generation using Gemini
8. Display of the generated answer to the user

The project uses a local Sentence Transformer model for generating embeddings and FAISS for efficient vector similarity search.

1.3 Purpose of the Document

This Software Requirements and Design Document describes the requirements, architecture, components, functionality, design, constraints, and testing requirements of the CampusGPT system.

The document acts as a technical reference for understanding how the system is expected to work and how its major components are connected.

2. PROBLEM STATEMENT

Students and users often need to search through lengthy PDF documents to find specific information.

Manually reading a large PDF can be time-consuming, especially when the user wants an answer to a specific question.

Traditional keyword-based searching may also be insufficient when the question and the information in the document use different wording.

Therefore, there is a need for a system that can:

- Accept PDF documents from users.
- Understand the textual content of the documents.
- Identify relevant portions of the document.
- Retrieve information related to a user's question.
- Generate a natural-language answer.
- Provide the answer through an easy-to-use web interface.

CampusGPT addresses this problem using Retrieval-Augmented Generation.

3. OBJECTIVES

The major objectives of CampusGPT are:

1. To develop a web-based PDF question-answering system.
2. To allow users to upload PDF documents.
3. To extract textual information from uploaded PDF files.
4. To divide extracted text into smaller manageable chunks.
5. To generate numerical vector representations of the text using embeddings.
6. To store and search these vectors using FAISS.
7. To retrieve the most relevant document content for a user question.
8. To use Gemini to generate a natural-language response based on retrieved context.
9. To provide a simple and user-friendly interface.
10. To reduce the time required to manually search large PDF documents.
11. To demonstrate the practical use of Retrieval-Augmented Generation in an educational application.

4. SCOPE OF THE SYSTEM

4.1 Included Scope

The current system includes:

- PDF document upload.
- PDF text extraction.
- Text preprocessing and chunking.
- Local text embedding generation.
- FAISS vector indexing.
- Similarity-based information retrieval.
- Gemini-based response generation.
- Web-based interaction through Flask.
- A browser-based user interface.
- Question and answer interaction with uploaded document content.

4.2 Out of Scope

The current version does not focus on:

- User account management.
- User authentication.
- Permanent cloud storage of uploaded documents.
- A relational database.
- Simultaneous processing of many uploaded PDFs as a persistent collection.
- Advanced document editing.
- Image-based OCR for scanned documents.
- Voice-based question answering.
- Mobile application development.

These features may be considered for future versions.

5. EXISTING SYSTEM

In a traditional approach, users generally open a PDF document and manually search for required information.

Some existing PDF readers provide keyword-based search functionality.

However, keyword search has limitations:

- Users must know the appropriate keywords.
- It may not understand the meaning of a question.
- Users may need to read multiple sections to understand the answer.
- Searching lengthy documents can be time-consuming.
- It does not normally provide a natural-language explanation.

Another approach is to directly ask a general-purpose AI model a question. However, the model may not have access to the specific content of the user's PDF.

6. PROPOSED SYSTEM

CampusGPT provides a Retrieval-Augmented Generation-based solution.

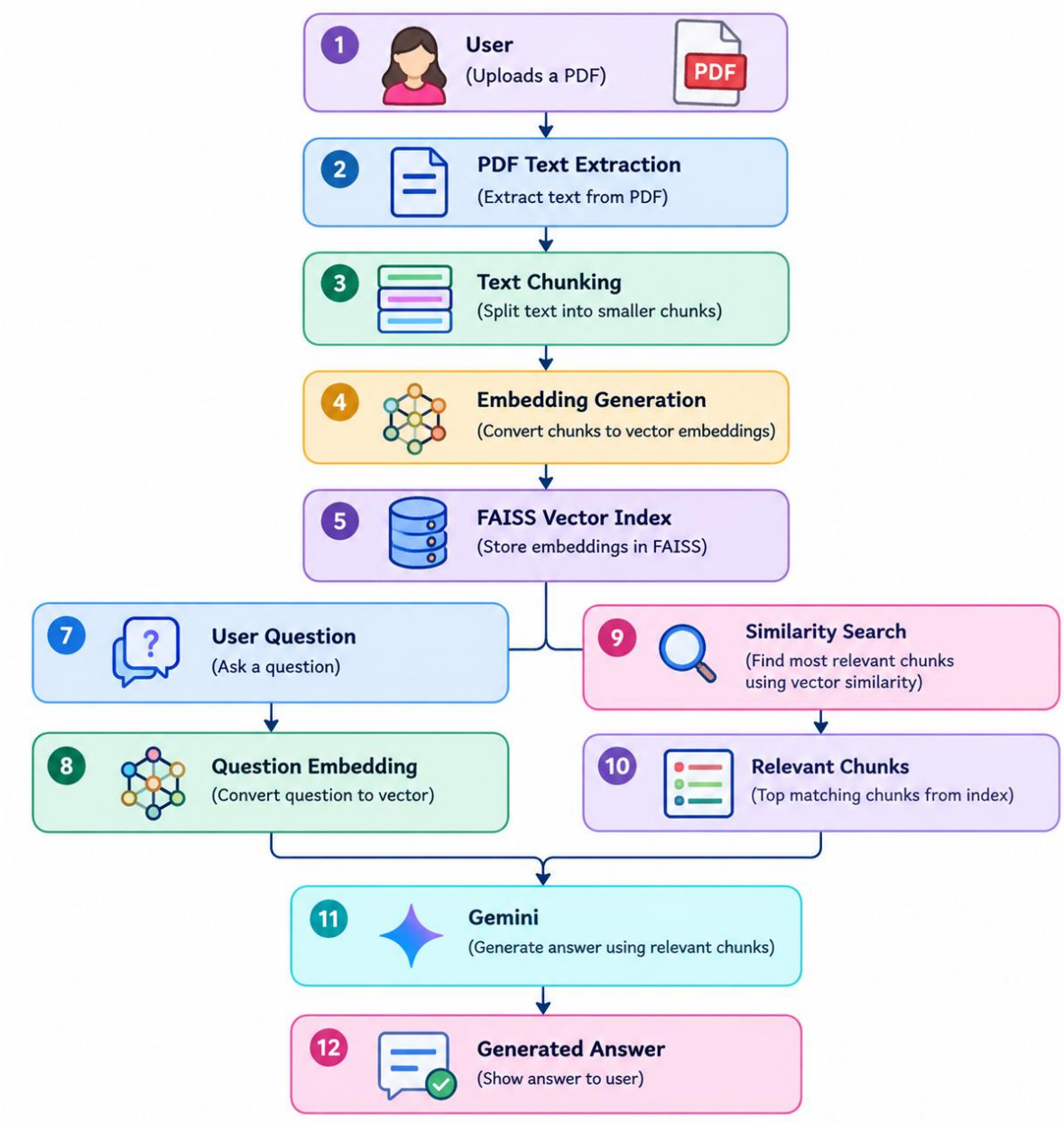
The proposed system allows a user to upload a PDF and ask questions related to its content.

The system first processes the document and creates searchable vector representations.

When the user asks a question, the system converts the question into an embedding and searches the FAISS index for relevant document chunks.

The retrieved information is then supplied to Gemini as context for generating the response.

Proposed workflow



7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

The following hardware is sufficient for development and basic operation:

Component	Requirement
Processor	Intel/AMD processor or equivalent
RAM	8 GB recommended
Storage	At least 5 GB free space
Display	Standard monitor
Internet	Required for Gemini API access
Input	Keyboard and mouse

The exact performance may vary depending on the PDF size, embedding model, and system configuration.

7.2 Software Requirements

Software	Purpose
Windows	Development operating system
Python 3.11	Application development
Visual Studio Code	Code development
Flask	Web application framework
FAISS	Vector similarity search
Sentence Transformers	Text embeddings
PyPDF	PDF text extraction
Google Gemini API	AI answer generation
python-dotenv	Environment variable management
HTML	Web page structure
CSS	Web page styling
JavaScript	Frontend interaction
Web Browser	Accessing the application

8. FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that the system must perform.

FR-01: PDF Upload

The system shall allow the user to select and upload a PDF document.

FR-02: PDF Validation

The system shall accept PDF documents for processing.

FR-03: Text Extraction

The system shall extract text from the pages of the uploaded PDF.

FR-04: Text Chunking

The system shall divide extracted text into smaller chunks before creating embeddings.

FR-05: Embedding Generation

The system shall convert text chunks into numerical vector representations using a Sentence Transformer embedding model.

FR-06: Vector Indexing

The system shall store the generated embeddings in a FAISS vector index.

FR-07: Question Processing

The system shall accept a question submitted by the user through the web interface.

FR-08: Question Embedding

The system shall convert the user's question into an embedding representation.

FR-09: Similarity Search

The system shall compare the question embedding with document embeddings and retrieve relevant chunks.

FR-10: Context Creation

The system shall use the retrieved document chunks as context for generating the answer.

FR-11: AI Answer Generation

The system shall send the relevant context and user question to the Gemini model to generate a response.

FR-12: Answer Display

The system shall display the generated response through the web interface.

FR-13: Invalid Question Handling

The system shall provide an appropriate response when relevant information cannot be found in the uploaded document.

FR-14: Chat Interaction

The interface shall allow the user to submit questions and receive responses through the chat-style interface.

9. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe qualities and constraints of the system rather than individual functions. Common categories include performance, usability, security, reliability, and scalability.

9.1 Usability

The application should provide a simple interface that can be used without technical knowledge.

Users should be able to:

- Upload a PDF.
- Enter a question.
- Submit the question.
- Read the generated answer.

9.2 Performance

The system should process user questions within a reasonable time under normal operating conditions.

Response time may depend on:

- PDF size.
- Number of chunks.
- Local embedding computation.
- Internet connection.
- Gemini API response time.

9.3 Reliability

The system should handle common errors without crashing the complete application.

Examples include:

- Invalid PDF files.
- Missing questions.
- Empty document content.
- API-related failures.

9.4 Security

The Gemini API key shall not be hard-coded directly into the source code.

The API key shall be stored in an environment configuration file.

Example:

GOOGLE_API_KEY=
"AQ.Ab8RN6Ir2laQ59tvYbTf99VhjOmGQWkn_3OYvpN2IVSsB8dZfw"

The .env file should not be shared publicly or uploaded to GitHub.

9.5 Maintainability

The application should be organized into separate components for:

- PDF processing
- Text chunking
- Embedding generation
- Vector search
- Answer generation
- Web interface

This structure makes the application easier to understand and modify.

9.6 Scalability

The current system is designed primarily as an academic/minor project.

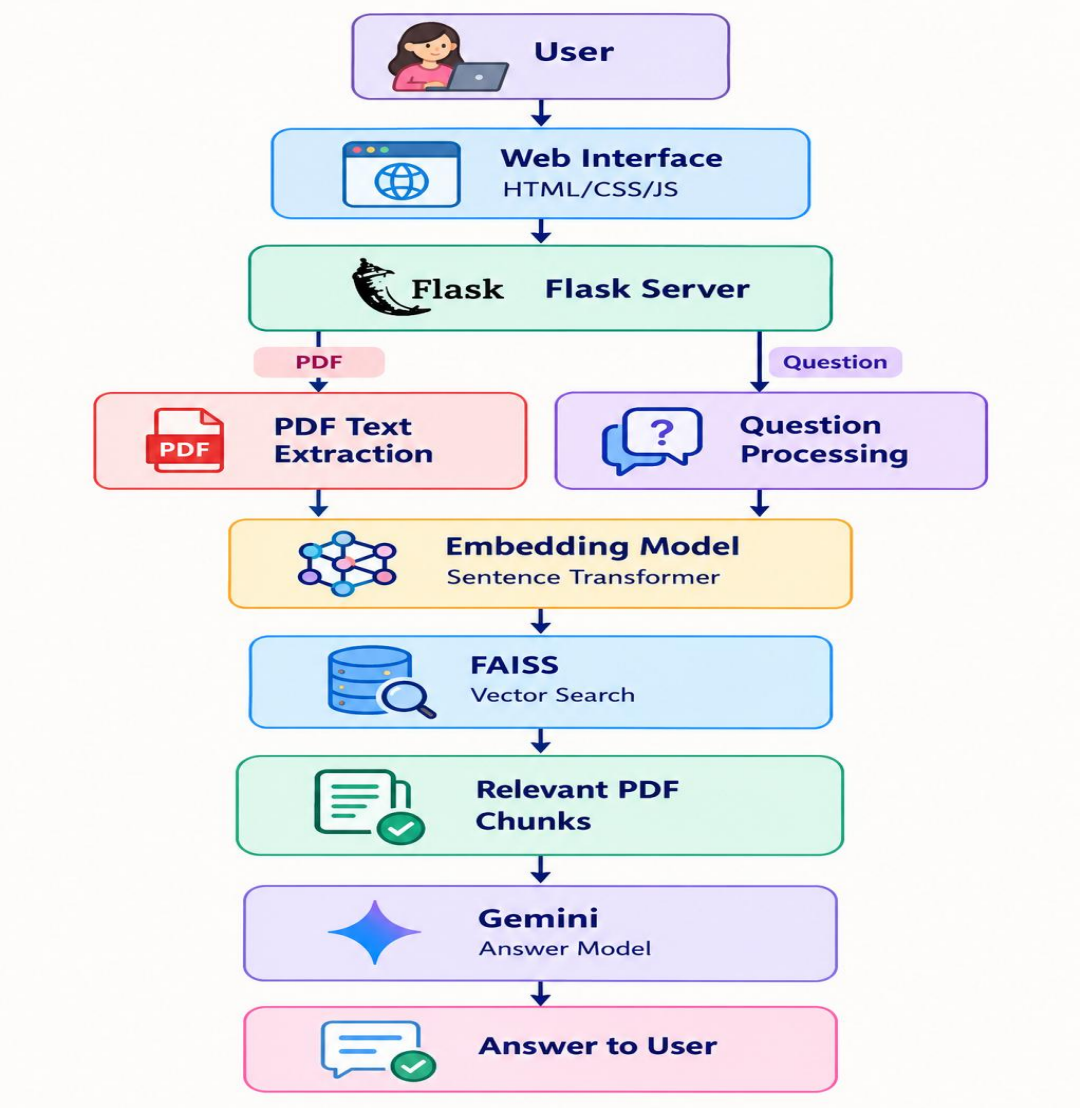
Future versions can improve scalability by using persistent vector databases, cloud storage, authentication, and optimized document processing.

10. SYSTEM ARCHITECTURE

CampusGPT follows a RAG-based architecture.

The major components are:

1. User Interface
2. Flask Application
3. PDF Processing Module
4. Text Chunking Module
5. Embedding Module
6. FAISS Vector Store
7. Retrieval Module
8. Gemini Generation Module



11. MODULE DESCRIPTION

11.1 User Interface Module

This module provides the graphical interface through which users interact with CampusGPT.

Main interface elements include:

- CampusGPT application header.
- PDF upload section.
- PDF filename display.
- Chat area.
- Question input box.
- Send button.
- Suggested questions.

11.2 PDF Processing Module

This module receives the uploaded PDF and extracts textual information from its pages.

The extracted information is associated with page information so that document content can be traced back to its source page during processing.

11.3 Text Chunking Module

Large extracted text is divided into smaller sections called chunks.

Chunking helps the retrieval system search smaller pieces of the document instead of processing the entire PDF for every question.

The current implementation uses configurable chunk size and overlap.

11.4 Embedding Module

The system uses the Sentence Transformer model:

all-MiniLM-L6-v2

The model converts text into numerical vectors.

These vectors represent the semantic meaning of the text and can be compared during similarity search.

11.5 FAISS Module

FAISS is used to create a vector index.

The index allows the system to search for document chunks that are semantically similar to the user's question.

11.6 Retrieval Module

The retrieval module:

1. Receives the user question.
2. Generates its embedding.
3. Searches the FAISS index.
4. Retrieves the most relevant chunks.
5. Provides those chunks to the answer-generation stage.

11.7 Gemini Generation Module

Gemini receives:

- User question
- Retrieved document context
- Relevant conversation information, where applicable

It then generates a natural-language response.

11.8 Flask Web Module

Flask connects the frontend with the Python backend.

Important routes include:

/

Used to display the main application page.

/ask

Used to process user questions.

/upload

Used to process uploaded PDF documents.

12. RETRIEVAL-AUGMENTED GENERATION WORKFLOW

Retrieval-Augmented Generation is the central concept of CampusGPT.

Step 1: Upload PDF

The user selects a PDF from their computer and uploads it.

Step 2: Extract Text

The system reads the PDF and extracts text from its pages.

Step 3: Create Chunks

The extracted text is divided into smaller overlapping chunks.

Step 4: Generate Embeddings

Each chunk is converted into a numerical vector using the Sentence Transformer model.

Step 5: Build FAISS Index

The generated vectors are stored in a FAISS index.

Step 6: Enter Question

The user asks a question through the web interface.

Step 7: Generate Question Embedding

The question is converted into a vector using the same embedding model.

Step 8: Retrieve Relevant Chunks

FAISS searches for the document chunks that are most similar to the question.

Step 9: Generate Answer

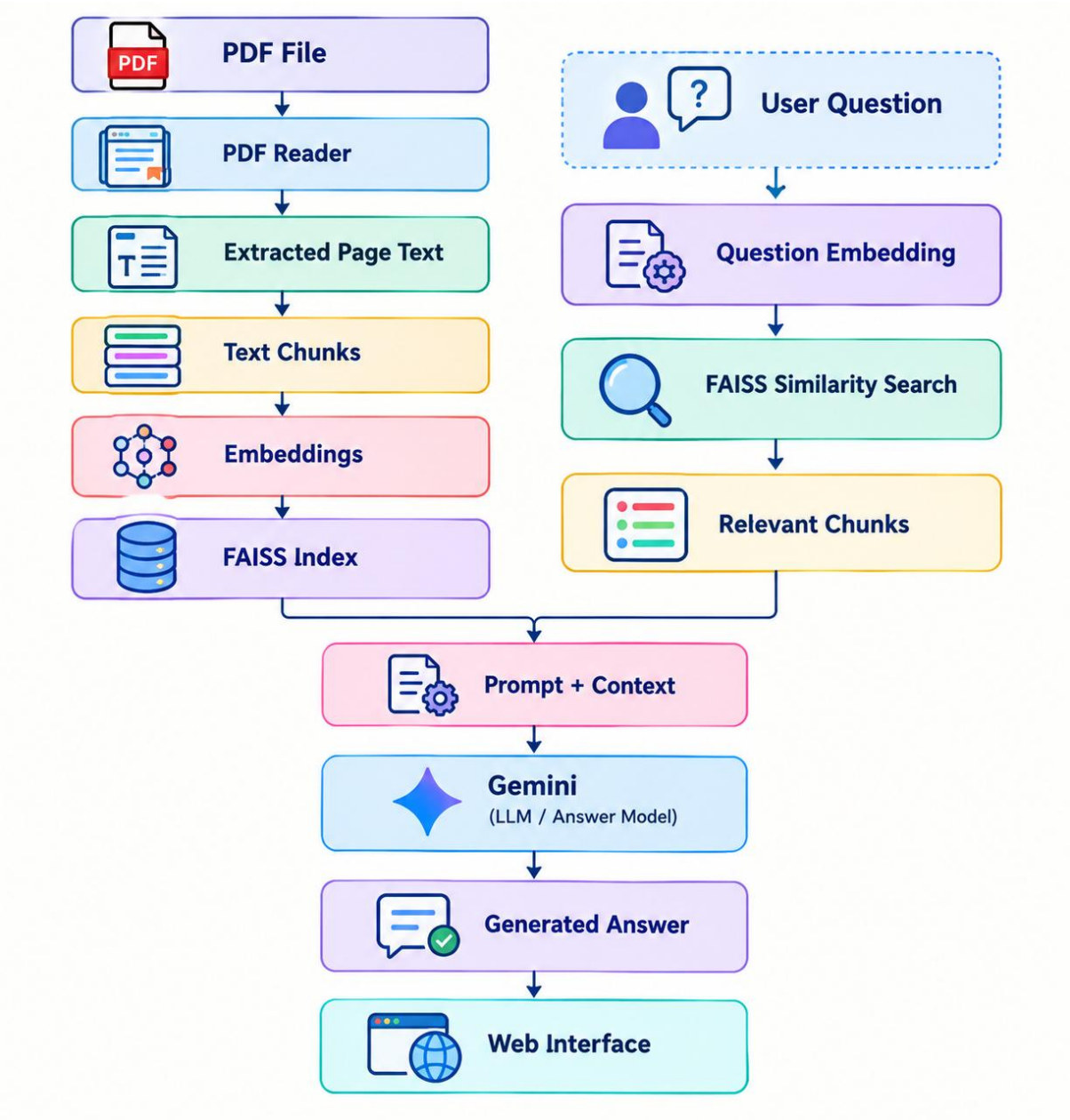
The retrieved content is provided to Gemini as context.

Step 10: Display Answer

The generated response is returned to the Flask application and displayed to the user.

13. DATA FLOW

The data flow of CampusGPT can be represented as:



14. USER INTERFACE REQUIREMENTS

The application should provide an interface that is simple and understandable.

14.1 Upload Interface

The upload section should allow the user to:

- Select a PDF.
- See the selected filename.
- Upload the document.
- Receive an indication that processing is taking place.

14.2 Chat Interface

The chat interface should allow the user to:

- Enter a question.
- Submit the question.
- View the generated answer.
- Continue asking questions.

14.3 Responsive Design

The interface should be usable on different screen sizes, including:

- Desktop
- Laptop
- Tablet
- Mobile-sized screens

15. INPUT AND OUTPUT

15.1 Inputs

The system accepts:

Document Input

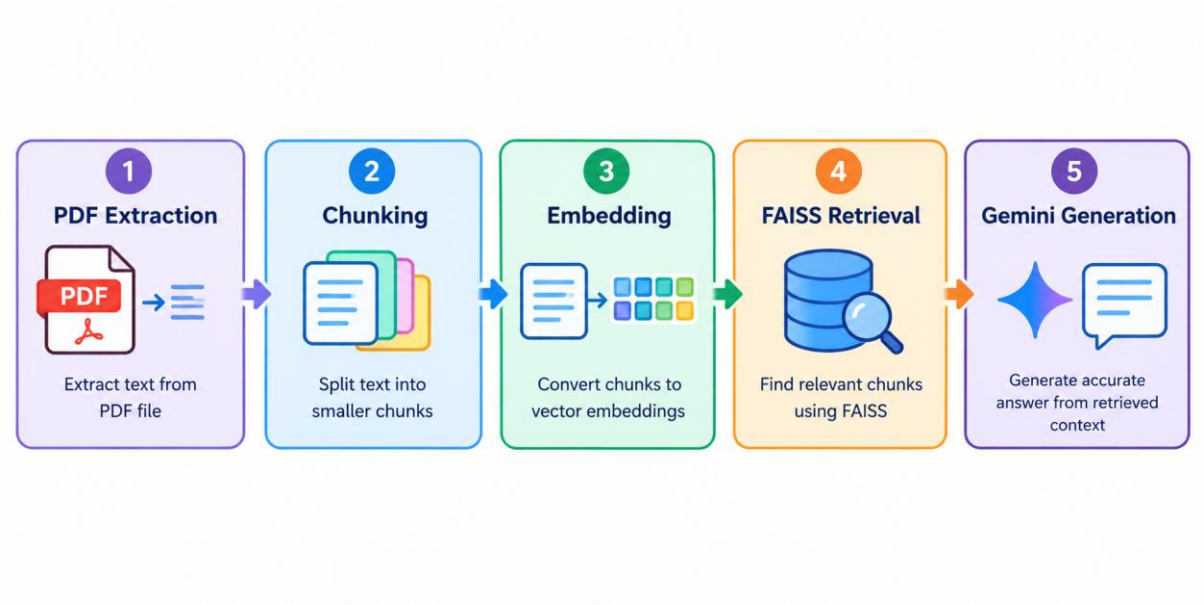
PDF file

User Input

Natural-language question

15.2 Processing

The system processes the inputs using:



15.3 Output

The system produces:

Natural-language answer

based on the retrieved document context.

16. ERROR HANDLING

The application should handle common errors gracefully.

16.1 Invalid PDF

If the selected file is not a valid PDF, the application should display an appropriate error message.

16.2 Empty PDF

If a PDF does not contain extractable text, the system should notify the user that useful content could not be extracted.

16.3 Empty Question

If the user submits an empty question, the system should request a valid question.

16.4 Information Not Found

If the required information cannot be found in the uploaded document, the system should provide an appropriate response instead of inventing information.

16.5 API Error

If communication with the Gemini API fails, the system should display a user-friendly error rather than exposing technical details unnecessarily.

17. SECURITY REQUIREMENTS

The system should follow basic security practices.

17.1 API Key Protection

The Gemini API key should be stored in the .env file.

It should not be directly written in the source code.

17.2 Environment File Protection

The .env file should be included in .gitignore.

Example:

.env

venv/

__pycache__/

*.pyc

17.3 File Upload Safety

Uploaded files should be handled carefully and should be restricted to the expected PDF format.

17.4 Secret Management

Sensitive credentials should not be included in publicly shared source code.

18. TECHNOLOGY STACK

Technology	**Role**
Python	Backend programming
Flask	Web application framework
HTML	Frontend structure
CSS	Frontend styling
JavaScript	Frontend interaction
PyPDF	PDF text extraction
Sentence Transformers	Text embeddings
all-MiniLM-L6-v2	Embedding model
FAISS	Vector similarity search
Gemini	AI response generation
python-dotenv	Environment configuration
VS Code	Development environment
Web Browser	Application access

19. PROJECT STRUCTURE

The current project follows a structure similar to:

Minor project/

```
│
├── venv/
│
├── .env
├── app.py
├── requirements.txt
├── sample.pdf
├── README.md
│
├── templates/
│   └── index.html
│
├── static/
│   └── style.css
│
└── uploads/
```

Description

File/Folder	Purpose
app.py	Main Flask application and RAG processing
.env	Stores configuration such as API key
requirements.txt	Python dependencies
sample.pdf	Sample document used for testing
README.md	Project documentation
templates/index.html	Web interface
static/style.css	Website styling

uploads/	Uploaded PDF files
venv/	Python virtual environment

20. DESIGN CONSIDERATIONS

The system is designed around the following principles:

20.1 Simplicity

The interface should be easy for students and general users to understand.

20.2 Modular Design

Different processing stages are separated logically so that individual components can be modified independently.

20.3 Semantic Retrieval

The system uses embeddings rather than relying only on exact keyword matching.

20.4 Context-Based Generation

The language model receives retrieved document information as context before generating an answer.

20.5 Local Embedding Processing

The embedding model is executed locally, while Gemini is used for language generation.

21. SYSTEM CONSTRAINTS

The system has the following constraints:

1. An internet connection is required for Gemini API communication.
2. Answer quality depends on the quality and content of the uploaded PDF.
3. PDFs containing primarily scanned images may not provide extractable text without OCR.
4. Processing time can increase with document size.
5. Gemini API usage may be subject to API limits and availability.
6. The current implementation is intended primarily for educational and demonstration purposes.
7. The system does not currently provide a complete user account and authentication system.

22. LIMITATIONS

The current version has several limitations:

- Text extraction depends on the PDF content being machine-readable.
- Scanned documents may require OCR.
- The system relies on an external Gemini API for answer generation.
- Large documents may require additional optimization.
- The current application is designed primarily for academic use.
- Advanced user authentication is not included.
- Persistent document management is not currently implemented.
- The quality of answers depends on the quality of retrieved context.

23. FUTURE ENHANCEMENTS

The following improvements can be considered in future versions:

1. Support for multiple PDFs simultaneously.
2. Persistent document storage.
3. User authentication and account management.
4. Chat history storage.
5. Improved source and page citation display.
6. OCR support for scanned PDFs.
7. Support for additional document formats.
8. Voice-based question input.
9. Improved retrieval techniques.
10. Reranking of retrieved documents.
11. Cloud deployment.
12. Mobile application support.
13. Database integration.
14. Improved analytics and usage monitoring.
15. More advanced document management.

24. TESTING REQUIREMENTS

The system should be tested using different types of inputs.

Test Case 1 — Application Launch

Input: Start the Flask application.

Expected Result:
The application should start successfully and be accessible through the local browser.

Test Case 2 — PDF Upload

Input: Upload a valid PDF.

Expected Result:
The PDF should be accepted and processed.

Test Case 3 — Valid Question

Input:

Ask a question whose answer is present in the PDF.

Expected Result:
The system should retrieve relevant content and generate an appropriate answer.

Test Case 4 — Invalid Question

Input:

Ask a question unrelated to the uploaded document.

Expected Result:
The system should indicate that the required information could not be found in the uploaded document.

Test Case 5 — Multiple Questions

Input: Ask several questions sequentially.

Expected Result:
The system should process each question and return responses.

Test Case 6 — Empty Question

Input: Submit an empty question.

Expected Result:
The application should prevent or appropriately handle the empty input.

Test Case 7 — Different PDF

Input: Upload another valid PDF.

Expected Result:
The system should process the new document and allow questions related to its content.

Test Case 8 — API Failure

Input: Simulate an unavailable or invalid API configuration.

Expected Result:
The application should display an understandable error instead of crashing unexpectedly.

25. ACCEPTANCE CRITERIA

The CampusGPT system will be considered functional when:

1. The application starts successfully.
2. A user can access the application through a web browser.
3. A PDF can be uploaded.
4. Text can be extracted from the PDF.
5. Text can be divided into chunks.
6. Embeddings can be generated.
7. A FAISS index can be created.
8. A user can enter a question.
9. Relevant document chunks can be retrieved.
10. Gemini can generate an answer using the retrieved context.
11. The answer can be displayed in the web interface.
12. The system can handle questions for which relevant information is unavailable.
13. Basic errors can be handled appropriately.

26. CONCLUSION

CampusGPT is a web-based AI learning assistant that uses Retrieval-Augmented Generation to answer questions based on uploaded PDF documents.

The system combines PDF text extraction, text chunking, semantic embeddings, FAISS vector search, and Gemini-based natural-language generation.

The proposed system reduces the effort required to manually search through lengthy documents and provides users with an interactive way to obtain information from their study materials.

The project demonstrates the practical application of Artificial Intelligence, Natural Language Processing, vector databases, information retrieval, and web development in an educational environment.

CampusGPT can be further extended with features such as multiple-document support, persistent chat history, OCR, authentication, cloud deployment, and mobile application support.

27. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my institution, faculty members, project guide, and everyone who provided guidance and support during the development of the CampusGPT project.

I am thankful for the resources, technical guidance, and learning opportunities that helped me understand concepts related to Artificial Intelligence, Natural Language Processing, Retrieval-Augmented Generation, vector search, and web application development.

I also acknowledge the open-source technologies and development tools used in this project, including Python, Flask, Sentence Transformers, FAISS, PyPDF, and other supporting technologies.